DBMS MINI PROJECT

Smart Inventory & Supply Chain Tracker

A multi-warehouse inventory and procurement system with supplier-quality analytics, built on PostgreSQL and MongoDB with a Next.js web interface.

PostgreSQL 18 | MongoDB Atlas | Next.js 16 | Option 1: SQL + NoSQL

TEAM MEMBER	SRN	PRIMARY RESPONSIBILITY
Rahul Prasanna	PES1UG24AM217	Next.js application, PostgreSQL integration, testing
Prithvish Adhikari	PES1UG24AM206	Database design, MongoDB, supplier-quality module

Repository: github.com/RahulPrasanna22/DBMS-Mini_Project

October 2026

Contents

1	Problem Statement	motivation and the problem being solved
2	Objectives	what the system sets out to achieve
3	Scope	in scope, out of scope, assumptions
4	Functional Requirements	features by role, non-functional requirements
5	Database Choice and Justification	why SQL and why NoSQL for each type of data
6	ER Diagram	entities, relationships, cardinalities
7	Relational Schema	ER-to-relational mapping, keys, normalisation
8	Data Dictionary	every table, column and constraint
9	SQL Implementation	DDL, indexes, triggers, procedures, views, full-text search
10	NoSQL Implementation	MongoDB data model, validators, indexes, retrieval
11	Important Queries	DML, joins, aggregates, subqueries, pipelines
12	System Architecture	layers, request flow, security
13	Application Screenshots	the Next.js interface
14	Testing Results	test cases and evidence
15	Individual Contributions	work split between team members
16	Conclusion and Future Enhancements	
A	Appendix: Setup and Execution	how to run the project

Requirement coverage. Sections 1–16 follow the documentation list in the project brief (item 14). The advanced database features required by item 7 are summarised in Section 9.7. How the brief's checklist is met is summarised in Section 16.1.

1 Problem Statement

Why the system is needed

Small and mid-sized distributors in India commonly hold stock in several warehouses in different cities and buy from many suppliers. In practice, their records are spread across spreadsheets, invoices and messaging apps. This causes four recurring problems:

1. **Stock is not visible across warehouses.** Staff cannot tell how much of a product is in each warehouse, so items run out in one city while lying unused in another.
2. **Stock changes are not reliable.** When stock is moved between warehouses or a delivery is received, two records have to change together. If only one is updated, the totals become wrong and nobody can trace why.
3. **Low stock is noticed too late.** Without automatic alerts, reordering depends on someone happening to look.
4. **Supplier quality is not measured.** Late or damaged deliveries are remembered informally, and supplier certificates (FSSAI, BIS, GST, ISO) expire without anyone noticing.

The data involved is also of two different kinds. Stock levels, orders and transfers are **highly structured and transactional**: they need exact arithmetic, referential integrity and atomic updates. Product specifications, supplier certificates and delivery feedback are **semi-structured**: a bag of rice and a USB cable have completely different attributes, and feedback is free text with a varying set of issue tags.

> **Problem statement.** Design and build a web-based inventory and supply-chain system that keeps stock consistent across multiple warehouses, enforces business rules inside the database, automatically flags low stock, manages the purchase-order lifecycle, and evaluates supplier quality. It should use a relational database for transactional data and a document database for flexible data, each chosen for the nature of the data it stores.

2 Objectives

What the project sets out to achieve

1. Design a normalised relational schema (3NF) for products, suppliers, warehouses, inventory, purchase orders and stock movements, with primary keys, foreign keys and CHECK constraints.
2. Enforce integrity **at the database level**: warehouse capacity, non-negative stock, valid order states and unique identifiers must hold even if the application has a bug.
3. Make multi-row operations (receiving an order, transferring stock) **atomic** using stored procedures that run as transactions with row-level locking.
4. Raise and resolve **low-stock alerts automatically** using triggers, and suggest reorder quantities through a view.
5. Provide fast product search using PostgreSQL **full-text search** with a GIN index.
6. Store category-specific product specifications and supplier compliance and feedback data in **MongoDB**, validated with `$jsonSchema`, and analyse it with **aggregation pipelines**.
7. Build a **Next.js** web interface with forms, tables, search, filters and dashboards, connected to both databases through an API layer.
8. Implement **authentication and role-based access** (Admin, Manager, Supplier) with bcrypt-hashed passwords and no hard-coded credentials.

3 Scope

Boundaries of the system

In scope

- Product catalogue with categories, suppliers, prices and reorder levels
- Multiple warehouses with a capacity limit each
- Per-warehouse inventory and an audit log of every stock movement
- Purchase orders: create (multi-item), receive, cancel
- Stock transfers between warehouses
- Automatic low-stock alerts and reorder suggestions
- Full-text product search, filters and sorting
- Dashboard analytics (stock value, utilisation, supplier volume)
- Category-specific product specifications (MongoDB)
- Supplier certificates, delivery feedback and scorecards (MongoDB)
- Login with three roles and role-based page/API access

Out of scope

- Sales orders, invoicing, payments and GST filing
- Barcode or RFID scanning hardware
- Shipment tracking and logistics routing
- Email or SMS notifications
- Multi-company (multi-tenant) operation
- Public deployment; the system runs locally with a cloud MongoDB

Assumptions

- Each product has one primary supplier.
- Warehouse capacity is measured in total units.
- A supplier user account is linked to exactly one supplier.
- Feedback is recorded only after an order is received.

4 Functional Requirements

What each type of user can do

4.1 User roles

Role	Description
ADMIN	Full access, including managing users, suppliers, warehouses and supplier certificates.
MANAGER	Day-to-day operations: products, purchase orders, transfers, alerts, supplier quality and delivery feedback.
SUPPLIER	External supplier login. Sees only their own products, purchase orders and documents.

4.2 Functional requirements

ID	Requirement	Roles
FR1	Users log in with username and password; sessions expire after 8 hours; users can log out.	All
FR2	Each page and API endpoint is restricted by role; unauthorised access shows a "forbidden" page or HTTP 403.	All
FR3	View, search (full-text), filter by category or supplier, and sort products.	All (suppliers: own only)
FR4	Add and edit products; the database rejects duplicate SKUs and negative prices.	Admin, Manager
FR5	Product detail page combines relational data, stock per warehouse and MongoDB specifications.	All
FR6	View inventory per warehouse and transfer stock between warehouses atomically.	Admin, Manager
FR7	Transfers are rejected if stock is insufficient or the destination warehouse would exceed its capacity.	Admin, Manager
FR8	Low-stock alerts are created automatically when stock falls to the reorder level, and resolved automatically when it recovers.	Admin, Manager
FR9	Reorder suggestions are listed, with a one-click "Create PO" action.	Admin, Manager

ID	Requirement	Roles
FR10	Create a multi-item purchase order in a single transaction; receive or cancel it. A PO cannot be received twice.	Admin, Manager
FR11	List purchase orders with totals, filtered by status.	All (suppliers: own only)
FR12	Dashboard: totals, stock value per category, warehouse utilisation, high-volume suppliers, never-ordered products, recent movements.	All (suppliers: own data)
FR13	Supplier-quality page: scorecard, most common delivery issues, expiring certificates, specification fields by category.	Admin, Manager
FR14	Record delivery feedback (rating 1–5, issue tags, comment) for a received purchase order, once per order.	Admin, Manager
FR15	Add supplier certificates with an expiry date.	Admin
FR16	Manage users (bcrypt-hashed passwords), suppliers (activate/deactivate) and warehouses.	Admin

4.3 Non-functional requirements

- **Integrity:** every business rule that can be expressed in the database is enforced there (constraints, triggers, validators), not only in the UI.
- **Consistency:** multi-row changes are atomic; concurrent transfers cannot deadlock or oversell stock.
- **Security:** no plain-text passwords, no credentials in source code, parameterised SQL only.
- **Resilience:** if MongoDB is unreachable, pages that also use PostgreSQL still show their relational data.
- **Usability:** database errors are shown to the user as readable messages.

5 Database Choice and Justification

Each database is chosen for the nature of the data it stores

The project uses **Option 1: SQL + NoSQL**. The rule followed was simple: if the data has a fixed structure, takes part in transactions or needs referential integrity, it goes to PostgreSQL. If its structure varies from record to record and it is read mostly as a whole document, it goes to MongoDB.

Data	Database	Reason
Users, roles	PostgreSQL	Fixed structure; the role CHECK constraint and the supplier foreign key must always hold.
Products, categories, suppliers, warehouses	PostgreSQL	Master data referenced by many tables; uniqueness (SKU, email) and foreign keys are essential.
Inventory, stock movements	PostgreSQL	Numeric quantities that must never go negative or exceed capacity; changed by multi-row transactions with row locks.
Purchase orders and items	PostgreSQL	Header and line items must be written together (ACID); status and date rules need CHECK constraints.
Low-stock alerts	PostgreSQL	Generated by triggers inside the same transaction as the stock change, so they can never be out of sync.
Product specifications	MongoDB	Attributes differ completely by category (rice: weight, grain, shelf life, nutrition; cable: connector, length, wattage). In SQL this would need many nullable columns or an EAV table. In MongoDB each product stores exactly its own fields, and a validator still enforces the required fields per category.
Supplier certificates, contacts	MongoDB	Certificate types vary by supplier (FSSAI for food, BIS for electronics, GST, ISO). Each certificate list is read and shown as a whole.
Delivery feedback	MongoDB	Free-text comments with a variable list of issue tags, growing over time as a nested array; analysed with aggregation pipelines (`$unwind` , `$group`).

How the two databases are linked

Each MongoDB document stores the PostgreSQL key it belongs to (`product_id` or `supplier_id`, both uniquely indexed). In the other direction, PostgreSQL stores the document's ObjectId in `products.mongo_spec_id` and `suppliers.mongo_doc_id`. MongoDB has no foreign keys, so cross-database rules are checked in the API layer. For example, feedback is only accepted for a purchase order that exists in PostgreSQL, belongs to that supplier and has status `RECEIVED`.

Why not vector search?

The brief lists vector search as optional. This project chose Option 1 because its unstructured data (specifications, certificates, feedback) is queried by exact fields and aggregated by tags and dates, not searched by meaning. Text search over products is handled by PostgreSQL full-text search. Section 16 lists semantic search over feedback as a future enhancement.

6 ER Diagram

Conceptual model in Chen notation: rectangles are entities, diamonds are relationships, ovals are relationship attributes

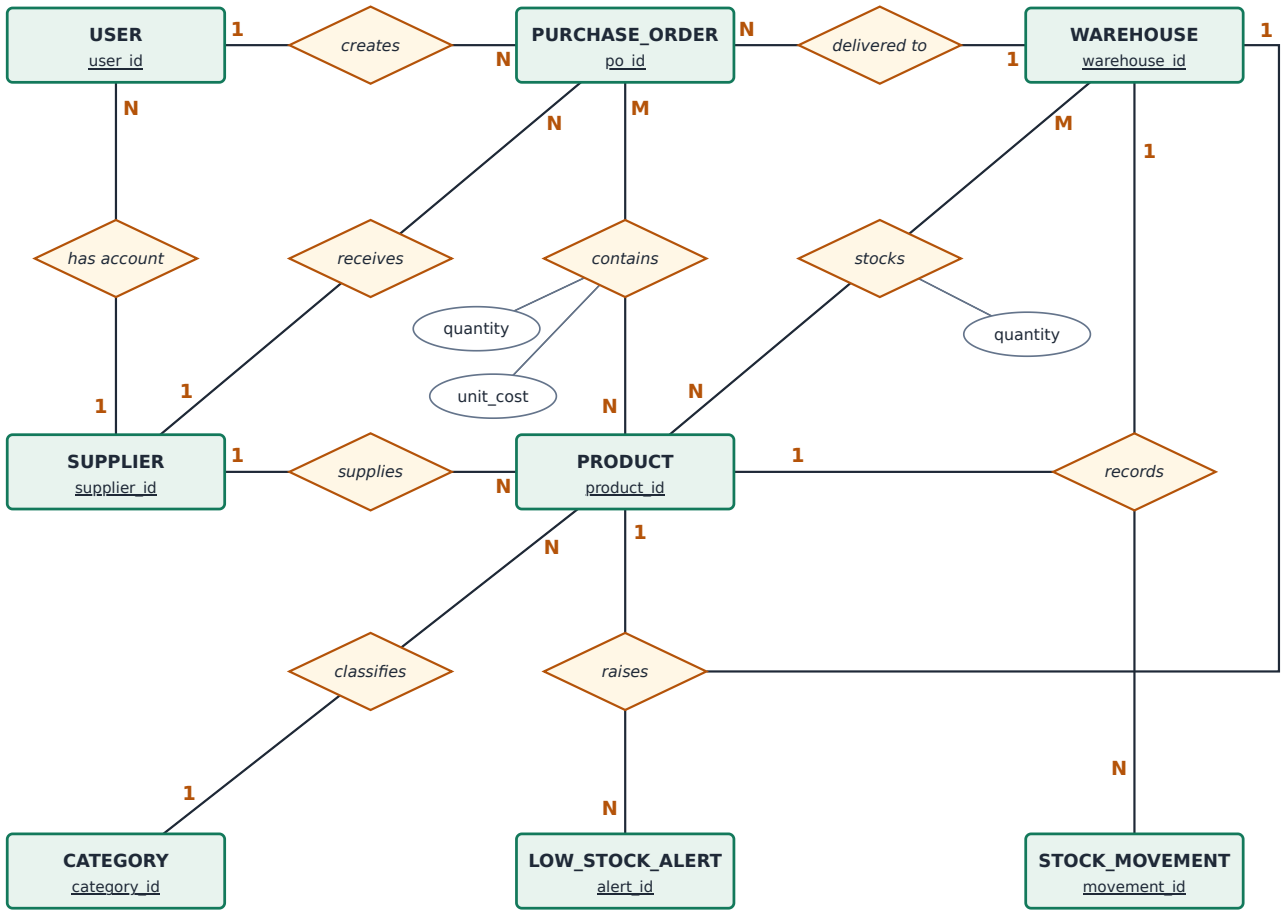


Figure 6.1: ER diagram of the relational part of the system (primary keys underlined)

6.1 Entities

Entity	Meaning
USER	A login account with a role (Admin, Manager or Supplier).
SUPPLIER	A company that supplies products.
CATEGORY	A product group, e.g. Grocery, Electronics.
PRODUCT	An item identified by a unique SKU.

Entity	Meaning
WAREHOUSE	A storage location with a maximum capacity in units.
PURCHASE_ORDER	An order placed with a supplier, delivered to one warehouse.
STOCK_MOVEMENT	An audit record of one change in stock (receipt, transfer in/out, adjustment).
LOW_STOCK_ALERT	Raised when a product's stock in a warehouse falls to its reorder level.

6.2 Relationships and cardinalities

Relationship	Cardinality	Description
SUPPLIER *supplies* PRODUCT	1 : N	Each product has one primary supplier.
CATEGORY *classifies* PRODUCT	1 : N	Each product belongs to one category.
SUPPLIER *has account* USER	1 : N (partial)	Only SUPPLIER-role users are linked to a supplier.
USER *creates* PURCHASE_ORDER	1 : N	The staff member who placed the order.
SUPPLIER *receives* PURCHASE_ORDER	1 : N	Each order is placed with one supplier.
WAREHOUSE *delivered to* PURCHASE_ORDER	1 : N	Each order is delivered to one warehouse.
PURCHASE_ORDER *contains* PRODUCT	M : N	An order has many products and a product appears in many orders. Attributes: *quantity, unit_cost*.
WAREHOUSE *stocks* PRODUCT	M : N	Attribute: *quantity* on hand.
records STOCK_MOVEMENT	1 : 1 : N	Each movement concerns one product in one warehouse.
raises LOW_STOCK_ALERT	1 : 1 : N	Each alert concerns one product in one warehouse.

The MongoDB documents are not part of the ER model, because they are not relational. Each PRODUCT has at most one `product_specs` document and each SUPPLIER at most one `supplier_documents` document (1 : 0..1). Section 10 describes them.

7 Relational Schema

Mapping the ER model to tables. **underlined** = primary key, *italic* = foreign key

USERS(**user_id**, username, password_hash, role, *supplier_id* → SUPPLIERS, created_at)
CATEGORIES(**category_id**, name)
SUPPLIERS(**supplier_id**, name, email, phone, city, rating, active, mongo_doc_id)
PRODUCTS(**product_id**, sku, name, description, *category_id* → CATEGORIES, *supplier_id* → SUPPLIERS, unit_price, reorder_level, mongo_spec_id, search)
WAREHOUSES(**warehouse_id**, name, city, capacity)
INVENTORY(***warehouse_id*** → WAREHOUSES, ***product_id*** → PRODUCTS, quantity, updated_at)
PURCHASE_ORDERS(**po_id**, *supplier_id* → SUPPLIERS, *warehouse_id* → WAREHOUSES, *created_by* → USERS, status, order_date, expected_date, received_date)
PO_ITEMS(***po_id*** → PURCHASE_ORDERS, ***product_id*** → PRODUCTS, quantity, unit_cost)
STOCK_MOVEMENTS(**movement_id**, *product_id* → PRODUCTS, *warehouse_id* → WAREHOUSES, change_qty, movement_type, reference_id, created_at)
LOW_STOCK_ALERTS(**alert_id**, *product_id* → PRODUCTS, *warehouse_id* → WAREHOUSES, quantity, resolved, created_at)

7.1 Mapping steps

1. **Strong entities → tables.** Each entity becomes a table with a surrogate primary key (`SERIAL` / `BIGSERIAL`). Natural candidate keys are kept as UNIQUE constraints: `sku`, `username`, supplier `email`, warehouse and category `name`.
2. **1 : N relationships → foreign keys** on the "many" side: `products.category_id`, `products.supplier_id`, `purchase_orders.supplier_id / warehouse_id / created_by`, `users.supplier_id`.
3. **M : N relationships → junction tables** with composite primary keys. *contains* becomes **PO_ITEMS** (PK `po_id, product_id`) carrying `quantity` and `unit_cost`; *stocks* becomes **INVENTORY** (PK `warehouse_id, product_id`) carrying `quantity`.
4. **Ternary relationships** (*records*, *raises*) become tables with two foreign keys: STOCK_MOVEMENTS and LOW_STOCK_ALERTS.
5. **Partial participation** (only supplier users have a supplier) becomes a nullable `users.supplier_id`, with a CHECK constraint so that `role = 'SUPPLIER'` exactly when `supplier_id` is set.

7.2 Normalisation

- **1NF:** all attributes are atomic. Lists such as a product's specifications or a supplier's certificates are not stored in PostgreSQL at all; they are in MongoDB.
- **2NF:** in the two tables with composite keys (INVENTORY, PO_ITEMS), every non-key attribute depends on the whole key. Stock quantity depends on (warehouse, product), and order quantity and cost depend on (order, product).
- **3NF:** there are no transitive dependencies. Supplier and category names are stored once and referenced by key; an order's total value is not stored but computed with `SUM(quantity × unit_cost)`, and stock value is computed in a view.
- **Deliberate exception:** `low_stock_alerts.quantity` is a snapshot of the stock level when the alert was raised. It is historical data, not a duplicate of current inventory.

8 Data Dictionary

PostgreSQL tables (database `inventory_db`)

users : application login accounts

Column	Type	Constraints / description
user_id	SERIAL	PRIMARY KEY
username	VARCHAR(50)	NOT NULL, UNIQUE
password_hash	TEXT	NOT NULL; bcrypt hash (cost 10), never the plain password
role	VARCHAR(20)	NOT NULL, CHECK IN ('ADMIN','MANAGER','SUPPLIER')
supplier_id	INT	FK → suppliers; CHECK: set if and only if role = 'SUPPLIER' (added with ALTER TABLE)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

categories : product groups

Column	Type	Constraints / description
category_id	SERIAL	PRIMARY KEY
name	VARCHAR(80)	NOT NULL, UNIQUE

suppliers : companies that supply products

Column	Type	Constraints / description
supplier_id	SERIAL	PRIMARY KEY
name	VARCHAR(120)	NOT NULL
email	VARCHAR(120)	NOT NULL, UNIQUE, CHECK (email LIKE '%_@_%._%')
phone	VARCHAR(20)	optional

Column	Type	Constraints / description
city	VARCHAR(80)	optional
rating	NUMERIC(2,1)	CHECK (rating BETWEEN 0 AND 5); kept in sync with the MongoDB feedback average
active	BOOLEAN	NOT NULL, DEFAULT TRUE (soft delete)
mongo_doc_id	VARCHAR(24)	ObjectId of the supplier's MongoDB document

products : the product catalogue

Column	Type	Constraints / description
product_id	SERIAL	PRIMARY KEY
sku	VARCHAR(30)	NOT NULL, UNIQUE
name	VARCHAR(150)	NOT NULL
description	TEXT	optional
category_id	INT	NOT NULL, FK → categories (indexed)
supplier_id	INT	NOT NULL, FK → suppliers (indexed)
unit_price	NUMERIC(10,2)	NOT NULL, CHECK (unit_price ≥ 0)
reorder_level	INT	NOT NULL, DEFAULT 10, CHECK (≥ 0); per-warehouse alert threshold
mongo_spec_id	VARCHAR(24)	ObjectId of the product's MongoDB specs document
search	TSVECTOR	GENERATED ALWAYS AS to_tsvector('english', name ‖ description) STORED; GIN index

warehouses : storage locations

Column	Type	Constraints / description
warehouse_id	SERIAL	PRIMARY KEY
name	VARCHAR(100)	NOT NULL, UNIQUE
city	VARCHAR(80)	NOT NULL
capacity	INT	NOT NULL, CHECK (capacity > 0); maximum total units, enforced by a trigger

inventory : stock on hand per warehouse and product (M:N junction)

Column	Type	Constraints / description
warehouse_id	INT	PK (part), FK → warehouses
product_id	INT	PK (part), FK → products
quantity	INT	NOT NULL, DEFAULT 0, CHECK (quantity ≥ 0)
updated_at	TIMESTAMPTZ	NOT NULL; set by the capacity trigger on every change

purchase_orders : orders placed with suppliers

Column	Type	Constraints / description
po_id	SERIAL	PRIMARY KEY
supplier_id	INT	NOT NULL, FK → suppliers
warehouse_id	INT	NOT NULL, FK → warehouses (delivery destination)
created_by	INT	NOT NULL, FK → users
status	VARCHAR(12)	NOT NULL, DEFAULT 'ORDERED', CHECK IN ('ORDERED','RECEIVED','CANCELLED'); composite index (status, supplier_id)
order_date	DATE	NOT NULL, DEFAULT CURRENT_DATE
expected_date	DATE	CHECK (expected_date IS NULL OR expected_date ≥ order_date)
received_date	DATE	CHECK ((status = 'RECEIVED') = (received_date IS NOT NULL))

po_items : line items of a purchase order (M:N junction)

Column	Type	Constraints / description
po_id	INT	PK (part), FK → purchase_orders ON DELETE CASCADE
product_id	INT	PK (part), FK → products
quantity	INT	NOT NULL, CHECK (quantity > 0)
unit_cost	NUMERIC(10,2)	NOT NULL, CHECK (unit_cost ≥ 0)

stock_movements : audit log of every stock change

Column	Type	Constraints / description
movement_id	BIGSERIAL	PRIMARY KEY
product_id	INT	NOT NULL, FK → products; index (product_id, created_at DESC)
warehouse_id	INT	NOT NULL, FK → warehouses
change_qty	INT	NOT NULL, CHECK (change_qty ≠ 0); positive = in, negative = out
movement_type	VARCHAR(15)	CHECK IN ('PO_RECEIPT','TRANSFER_IN','TRANSFER_OUT','ADJUSTMENT')
reference_id	INT	related PO number for receipts
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

low_stock_alerts : created and resolved by a trigger

Column	Type	Constraints / description
alert_id	SERIAL	PRIMARY KEY
product_id	INT	NOT NULL, FK → products; partial index WHERE resolved = FALSE
warehouse_id	INT	NOT NULL, FK → warehouses
quantity	INT	NOT NULL; stock level when the alert was raised
resolved	BOOLEAN	NOT NULL, DEFAULT FALSE; set TRUE by the trigger when stock recovers
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

8.1 MongoDB collections (database `supply_chain`)

product_specs : one document per product

Field	Type	Constraints / description
_id	ObjectId	stored in products.mongo_spec_id
product_id	int	required, ≥ 1, UNIQUE index; links to products.product_id
sku	string	required, UNIQUE index
category	string	required; index; decides which spec fields are required
specs	object	required. Grocery must have net_weight_kg, shelf_life_days, storage. Electronics must have warranty_months, certifications.
images	array<string>	optional
updated_at	date	optional

supplier_documents : one document per supplier

Field	Type	Constraints / description
_id	ObjectId	stored in suppliers.mongo_doc_id
supplier_id	int	required, ≥ 1, UNIQUE index; links to suppliers.supplier_id
certifications[]	array<object>	required; each {type, number, valid_until: date} required; index on valid_until
contacts[]	array<object>	free-form {name, role, phone, email}

Field	Type	Constraints / description
feedback[]	array<object>	required; each {po_id: int, rating: int 1–5, issues: [string], comment, date}; multikey index on feedback.issues

9 SQL Implementation

PostgreSQL 18: file `db/schema.sql` (DDL) and `db/seed_and_test.sql` (DML and tests)

9.1 DDL: CREATE, ALTER, DROP

The schema script is re-runnable. It first **drops** and recreates the schema, then **creates** the tables. Because `users` and `suppliers` reference each other in the design, the supplier foreign key and the role rule are added afterwards with **ALTER TABLE**.

```
DROP SCHEMA IF EXISTS public CASCADE;
CREATE SCHEMA public;

CREATE TABLE users (
    user_id        SERIAL PRIMARY KEY,
    username       VARCHAR(50)  NOT NULL UNIQUE,
    password_hash  TEXT         NOT NULL,                -- bcrypt hash, never plain text
    role           VARCHAR(20)  NOT NULL CHECK (role IN ('ADMIN','MANAGER','SUPPLIER')),
    supplier_id    INT,
    created_at     TIMESTAMPTZ  NOT NULL DEFAULT now()
);

ALTER TABLE users
    ADD CONSTRAINT fk_users_supplier FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id),
    ADD CONSTRAINT chk_supplier_role CHECK ((role = 'SUPPLIER') = (supplier_id IS NOT NULL));

CREATE TABLE products (
    product_id     SERIAL PRIMARY KEY,
    sku            VARCHAR(30)  NOT NULL UNIQUE,
    ...
    unit_price     NUMERIC(10,2) NOT NULL CHECK (unit_price >= 0),
    search         tsvector GENERATED ALWAYS AS
                   (to_tsvector('english', coalesce(name,'') || ' ' || coalesce(description,''))) STORED
);

CREATE TABLE purchase_orders (
    ...
    status   VARCHAR(12) NOT NULL DEFAULT 'ORDERED' CHECK (status IN ('ORDERED','RECEIVED','CANCELLED')),
    CHECK (expected_date IS NULL OR expected_date >= order_date),
    CHECK ((status = 'RECEIVED') = (received_date IS NOT NULL))
);
```

9.2 Indexes

Index	Type	Purpose
`idx_products_search`	GIN on tsvector	Full-text product search
`idx_products_category`, `idx_products_supplier`	B-tree	Joins and filters on foreign keys
`idx_po_status`	Composite B-tree	Purchase orders filtered by status and supplier
`idx_movements_product`	Composite, DESC	Latest movements of a product
`idx_alerts_open`	Partial (WHERE resolved = FALSE)	Fast lookup of open alerts only; stays small

9.3 Triggers

Trigger 1: warehouse capacity (BEFORE INSERT OR UPDATE on inventory)

Before any inventory row changes, the trigger adds up the warehouse's other stock and rejects the change if the total would exceed capacity. Because it runs inside the database, no application path can overfill a warehouse.

```
CREATE FUNCTION check_warehouse_capacity() RETURNS trigger AS $$
DECLARE v_cap INT; v_used INT;
BEGIN
    SELECT capacity INTO v_cap FROM warehouses WHERE warehouse_id = NEW.warehouse_id;
    SELECT COALESCE(SUM(quantity),0) INTO v_used
      FROM inventory WHERE warehouse_id = NEW.warehouse_id AND product_id <> NEW.product_id;
    IF v_used + NEW.quantity > v_cap THEN
        RAISE EXCEPTION 'Warehouse % capacity exceeded (% + % > %)', NEW.warehouse_id, v_used,
NEW.quantity, v_cap;
    END IF;
    NEW.updated_at := now();
    RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_capacity BEFORE INSERT OR UPDATE ON inventory
FOR EACH ROW EXECUTE FUNCTION check_warehouse_capacity();
```

Trigger 2: low-stock alerts (AFTER INSERT OR UPDATE OF quantity on inventory)

When stock crosses down to the reorder level, an alert is inserted. When it rises above the level again, open alerts for that product and warehouse are marked resolved. The `TG_OP = 'INSERT'` check also covers a transfer that creates a new, already-low inventory row.

```
CREATE FUNCTION raise_low_stock_alert() RETURNS trigger AS $$
DECLARE v_level INT;
BEGIN
    SELECT reorder_level INTO v_level FROM products WHERE product_id = NEW.product_id;
    IF NEW.quantity <= v_level AND (TG_OP = 'INSERT' OR OLD.quantity > v_level) THEN
        INSERT INTO low_stock_alerts(product_id, warehouse_id, quantity)
        VALUES (NEW.product_id, NEW.warehouse_id, NEW.quantity);
    ELSIF NEW.quantity > v_level THEN
        UPDATE low_stock_alerts SET resolved = TRUE
         WHERE product_id = NEW.product_id AND warehouse_id = NEW.warehouse_id AND NOT resolved;
    END IF;
    RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_low_stock AFTER INSERT OR UPDATE OF quantity ON inventory
FOR EACH ROW EXECUTE FUNCTION raise_low_stock_alert();
```

9.4 Stored procedures and transactions

Both procedures run as a single transaction: if any statement fails (including a trigger raising an exception), every change is rolled back.

receive_purchase_order(po_id)

Locks the order row with `SELECT … FOR UPDATE`, so two users clicking "Receive" at the same moment cannot both succeed. It then adds each item to inventory (an upsert with `ON CONFLICT`), logs a `PO_RECEIPT` movement and marks the order RECEIVED.

```
CREATE PROCEDURE receive_purchase_order(p_po_id INT) LANGUAGE plpgsql AS $$
DECLARE v_status TEXT; v_wh INT; r RECORD;
BEGIN
    SELECT status, warehouse_id INTO v_status, v_wh
      FROM purchase_orders WHERE po_id = p_po_id FOR UPDATE;          -- lock the PO row
    IF NOT FOUND THEN RAISE EXCEPTION 'PO % not found', p_po_id; END IF;
    IF v_status <> 'ORDERED' THEN RAISE EXCEPTION 'PO % is already %', p_po_id, v_status; END IF;

    FOR r IN SELECT product_id, quantity FROM po_items WHERE po_id = p_po_id LOOP
        INSERT INTO inventory(warehouse_id, product_id, quantity) VALUES (v_wh, r.product_id, r.quantity)
        ON CONFLICT (warehouse_id, product_id) DO UPDATE SET quantity = inventory.quantity +
EXCLUDED.quantity;
        INSERT INTO stock_movements(product_id, warehouse_id, change_qty, movement_type, reference_id)
        VALUES (r.product_id, v_wh, r.quantity, 'PO_RECEIPT', p_po_id);
    END LOOP;
    UPDATE purchase_orders SET status = 'RECEIVED', received_date = CURRENT_DATE WHERE po_id = p_po_id;
END $$;
```

transfer_stock(product, from, to, qty)

Locks both inventory rows **in a fixed order** (`ORDER BY warehouse_id`). Two transfers in opposite directions therefore always lock in the same sequence and cannot deadlock. The procedure then checks stock, moves it and logs both sides. A capacity violation in the destination raises an exception from the trigger and rolls back the whole transfer.

```
CREATE PROCEDURE transfer_stock(p_product INT, p_from INT, p_to INT, p_qty INT) LANGUAGE plpgsql AS $$
DECLARE v_have INT;
BEGIN
    IF p_from = p_to THEN RAISE EXCEPTION 'Source and destination must differ'; END IF;
    IF p_qty <= 0 THEN RAISE EXCEPTION 'Quantity must be positive'; END IF;
    PERFORM 1 FROM inventory WHERE product_id = p_product AND warehouse_id IN (p_from, p_to)
     ORDER BY warehouse_id FOR UPDATE;                        -- fixed lock order: no deadlocks
    SELECT quantity INTO v_have FROM inventory WHERE product_id = p_product AND warehouse_id = p_from;
    IF v_have IS NULL OR v_have < p_qty THEN
        RAISE EXCEPTION 'Insufficient stock: have %, need %', COALESCE(v_have,0), p_qty;
    END IF;
    UPDATE inventory SET quantity = quantity - p_qty WHERE product_id = p_product AND warehouse_id =
p_from;
    INSERT INTO inventory(warehouse_id, product_id, quantity) VALUES (p_to, p_product, p_qty)
    ON CONFLICT (warehouse_id, product_id) DO UPDATE SET quantity = inventory.quantity + EXCLUDED.quantity;
    INSERT INTO stock_movements(product_id, warehouse_id, change_qty, movement_type)
    VALUES (p_product, p_from, -p_qty, 'TRANSFER_OUT'), (p_product, p_to, p_qty, 'TRANSFER_IN');
END $$;
```

Application-level transaction: creating a purchase order

Creating an order writes one header row and several item rows. The API takes one dedicated connection from the pool and wraps them in `BEGIN … COMMIT`, with `ROLLBACK` on any error, so an order is never saved without its items.

```
const client = await pool.connect();
try {
  await client.query("BEGIN");
  const po = await client.query(`INSERT INTO purchase_orders (supplier_id, warehouse_id, created_by,
status, order_date, expected_date)
                                 VALUES ($1, $2, $3, 'ORDERED', CURRENT_DATE, $4) RETURNING po_id`, [...]);
  for (const it of items)
    await client.query(`INSERT INTO po_items (po_id, product_id, quantity, unit_cost) VALUES ($1, $2, $3,
$4)`, [...]);
  await client.query("COMMIT");
} catch (e) {
  await client.query("ROLLBACK");                 // nothing is half-saved
} finally { client.release(); }
```

9.5 Views

```
CREATE VIEW v_stock_summary AS                       -- total units and value per product
SELECT p.product_id, p.sku, p.name, c.name AS category,
       COALESCE(SUM(i.quantity),0) AS total_stock,
       COALESCE(SUM(i.quantity),0) * p.unit_price AS stock_value
FROM products p JOIN categories c ON c.category_id = p.category_id
LEFT JOIN inventory i ON i.product_id = p.product_id
GROUP BY p.product_id, p.sku, p.name, c.name, p.unit_price;

CREATE VIEW v_reorder_suggestions AS                 -- what to reorder, how much, from whom
SELECT i.warehouse_id, w.name AS warehouse, p.product_id, p.sku, p.name, i.quantity, p.reorder_level,
       (p.reorder_level * 2 - i.quantity) AS suggested_order_qty, s.supplier_id, s.name AS supplier
FROM inventory i
JOIN products p ON p.product_id = i.product_id JOIN warehouses w ON w.warehouse_id = i.warehouse_id
JOIN suppliers s ON s.supplier_id = p.supplier_id
WHERE i.quantity <= p.reorder_level;
```

9.6 Full-text search

The `search` column is a **generated** `tsvector` built from the name and description, so it never goes out of date. A GIN index makes matching fast, and results are ranked by relevance. The `english` configuration applies stemming, so *charging cable* matches "*Braided fast charging cable*".

```
SELECT p.sku, p.name, ts_rank(p.search, plainto_tsquery('english', $1)) AS relevance
FROM products p
WHERE p.search @@ plainto_tsquery('english', $1)
ORDER BY relevance DESC;
```

9.7 Advanced features: summary

The brief requires at least four advanced SQL features and one NoSQL feature. This project implements the following:

#	Feature	Where
1	Transactions	`receive_purchase_order`, `transfer_stock` (row locks, fixed lock order), PO creation with BEGIN/COMMIT/ROLLBACK
2	Stored procedures	`receive_purchase_order`, `transfer_stock` (PL/pgSQL)
3	Triggers	`trg_capacity` (BEFORE), `trg_low_stock` (AFTER)
4	Views	`v_stock_summary`, `v_reorder_suggestions`
5	Indexing	GIN, composite, descending and partial indexes
6	Full-text search	Generated tsvector, `plainto_tsquery`, `ts_rank`
7	NoSQL aggregation	4 MongoDB pipelines (Section 10.4)
8	NoSQL schema validation	`$jsonSchema` with per-category `oneOf` rules

10 NoSQL Implementation

MongoDB Atlas (free M0 cluster, Mumbai region): files `mongo/setup.mjs`, `web/lib/pipelines.js`

10.1 Data model

Two collections, each with one document per PostgreSQL row it describes. Embedding is used throughout: certificates, contacts and feedback are arrays inside the supplier's document, because they are always read together with the supplier and never on their own.

```
// product_specs: Grocery
{
  product_id: 1, sku: "G-001",
  category: "Grocery",
  specs: {
    net_weight_kg: 5, grain: "Long grain",
    aged_months: 12, shelf_life_days: 365,
    storage: "Cool, dry place",
    fssai_license: "10012345000123",
    nutrition_per_100g: {
      energy_kcal: 350, protein_g: 7.5,
      carbs_g: 78 }
  },
  images: ["rice-front.jpg"]
}
```

```
// product_specs: Electronics
{
  product_id: 2, sku: "E-001",
  category: "Electronics",
  specs: {
    connector: "USB-C to USB-C",
    length_m: 1.5, max_wattage: 60,
    data_rate_gbps: 0.48, braided: true,
    warranty_months: 12,
    certifications: ["BIS", "RoHS"]
  },
  images: ["cable-coil.jpg"]
}
```

```
// supplier_documents
{
  supplier_id: 2,
  certifications: [ { type: "BIS", number: "R-41012345", valid_until: ISODate("2026-11-21") },
                    { type: "GST", number: "33ABCDE1234F1Z5", valid_until: ISODate("2029-07-23") } ],
  contacts: [ { name: "S. Iyer", role: "Sales", email: "sales@globex.com" } ],
  feedback: [ { po_id: 4, rating: 3, issues: ["late_delivery"], comment: "Late by a week", date:
ISODate(...) } ]
}
```

10.2 Validation inside MongoDB

Both collections have a `$jsonSchema` validator with `validationAction: "error"`, so invalid documents are rejected by the database. The specs validator uses `oneOf`, so the **shape is flexible but still enforced per category**:

```
validator: { $jsonSchema: {
  bsonType: "object", required: ["product_id", "sku", "category", "specs"],
  properties: { product_id: { bsonType: "int", minimum: 1 }, ... },
  oneOf: [
    { properties: { category: { enum: ["Grocery"] },
                    specs: { required: ["net_weight_kg", "shelf_life_days", "storage"] } } },
    { properties: { category: { enum: ["Electronics"] },
                    specs: { required: ["warranty_months", "certifications"] } } }
  ] } }
// supplier_documents: feedback.rating must be { bsonType: "int", minimum: 1, maximum: 5 }
```

When MongoDB rejects a document (error code 121), the API returns the validator's message to the UI, in the same way PostgreSQL constraint errors are shown.

10.3 Indexes

Collection	Index	Purpose
product_specs	`{ product_id: 1 }` unique	Look up specs from a PostgreSQL product
product_specs	`{ sku: 1 }` unique	Look up by SKU
product_specs	`{ category: 1 }`	Filter by category
supplier_documents	`{ supplier_id: 1 }` unique	One document per supplier
supplier_documents	`{ "certifications.valid_until": 1 }`	Expiring-certificate queries
supplier_documents	`{ "feedback.issues": 1 }` multikey	Queries by issue tag

10.4 Aggregation pipelines

Pipeline 1: supplier scorecard

```
[ { $unwind: "$feedback" },
  { $group: { _id: "$supplier_id",
              avg_rating:  { $avg: "$feedback.rating" },
              reviews:     { $sum: 1 },
              issue_count: { $sum: { $size: { $ifNull: ["$feedback.issues", []] } } },
              last_feedback: { $max: "$feedback.date" } } },
  { $project: { _id: 0, supplier_id: "$_id", avg_rating: { $round: ["$avg_rating", 2] }, reviews: 1,
issue_count: 1, last_feedback: 1 } },
  { $sort: { avg_rating: 1 } } ]                      // worst suppliers first
```

Pipeline 2: most common delivery issues

```
[ { $unwind: "$feedback" }, { $unwind: "$feedback.issues" },
  { $group: { _id: "$feedback.issues", occurrences: { $sum: 1 }, suppliers: { $addToSet: "$supplier_id" } }
},
  { $sort: { occurrences: -1 } } ]
```

Pipeline 3: certificates expiring within N days

```
[ { $unwind: "$certifications" },
  { $match: { "certifications.valid_until": { $lte: new Date(Date.now() + days * 86400000) } } },
  { $project: { supplier_id: 1, type: "$certifications.type", number: "$certifications.number",
                days_left: { $dateDiff: { startDate: "$$NOW", endDate: "$certifications.valid_until", unit:
"day" } } } },
  { $sort: { days_left: 1 } } ]
```

Pipeline 4: specification fields by category

```
[ { $project: { category: 1, product_id: 1, fields: { $map: { input: { $objectToArray: "$specs" }, as: "f",
in: "$$f.k" } } } },
  { $unwind: "$fields" },
  { $group: { _id: "$category", spec_fields: { $addToSet: "$fields" }, products: { $addToSet: "$product_id"
} } },
  { $project: { category: "$_id", products: 1, spec_fields: { $sortArray: { input: "$spec_fields", sortBy:
1 } } } } ]
```

10.5 Retrieval strategy

1. **Product page:** the product row and per-warehouse stock come from PostgreSQL. Then `product_specs.findOne({ product_id })` fetches the specs, using the unique index.
2. **Supplier page:** the supplier row and its RECEIVED purchase orders come from PostgreSQL. The supplier document and a per-supplier scorecard (pipeline 1 with an extra first `$match`) come from MongoDB. The page then marks which orders already have feedback.
3. **Recording feedback:** the API checks in PostgreSQL that the order belongs to the supplier and is RECEIVED. It then `$push`es into `feedback` with `upsert: true`, recomputes the average and writes it back to `suppliers.rating`. If the supplier had no document yet, the new ObjectId is stored in `suppliers.mongo_doc_id`.
4. **Resilience:** if MongoDB cannot be reached, product and supplier pages still show their PostgreSQL data with a warning.

11 Important Queries

Representative DML and analytical queries used by the application (all parameterised)

11.1 DML: INSERT, UPDATE, DELETE, SELECT

```
-- INSERT with RETURNING (add a supplier; email format, uniqueness and rating are checked by the DB)
INSERT INTO suppliers (name, email, phone, city, rating, active)
VALUES ($1, $2, $3, $4, COALESCE($5::numeric, 0), true) RETURNING supplier_id, name;

-- UPDATE with COALESCE (partial edit / soft delete of a supplier)
UPDATE suppliers SET name = COALESCE($1, name), email = COALESCE($2, email), active = COALESCE($6::boolean,
active)
WHERE supplier_id = $7;

-- UPSERT (INSERT ... ON CONFLICT) inside the procedures
INSERT INTO inventory(warehouse_id, product_id, quantity) VALUES ($1, $2, $3)
ON CONFLICT (warehouse_id, product_id) DO UPDATE SET quantity = inventory.quantity + EXCLUDED.quantity;

-- Soft delete: the application never hard-deletes business records, so history is kept
UPDATE suppliers SET active = false WHERE supplier_id = $1;                        -- deactivate a supplier
UPDATE purchase_orders SET status = 'CANCELLED' WHERE po_id = $1 AND status = 'ORDERED';  -- cancel a PO

-- Hard DELETE (SQL level): po_items rows go automatically through ON DELETE CASCADE
DELETE FROM purchase_orders WHERE po_id = $1 AND status = 'CANCELLED';
```

Business records are deactivated or cancelled rather than deleted, so stock history and order history stay intact for auditing. Physical deletion is supported at the SQL level through `ON DELETE CASCADE` on `po_items`.

11.2 JOIN + GROUP BY + aggregate: stock value per category

```
SELECT v.category, SUM(v.stock_value) AS stock_value, SUM(v.total_stock)::int AS units
FROM v_stock_summary v
JOIN products p ON p.product_id = v.product_id
WHERE ($1::int IS NULL OR p.supplier_id = $1)          -- suppliers see only their own data
GROUP BY v.category
ORDER BY stock_value DESC;
```

11.3 LEFT JOIN + computed ratio: warehouse utilisation

```
SELECT w.warehouse_id, w.name, w.capacity,
       COALESCE(SUM(i.quantity), 0)::int AS used,
       ROUND(100.0 * COALESCE(SUM(i.quantity), 0) / w.capacity, 1) AS utilisation_pct
FROM warehouses w
LEFT JOIN inventory i ON i.warehouse_id = w.warehouse_id     -- empty warehouses still appear
GROUP BY w.warehouse_id, w.name, w.capacity
ORDER BY utilisation_pct DESC;
```

11.4 GROUP BY … HAVING: suppliers with more than N ordered units

```
SELECT s.supplier_id, s.name, SUM(i.quantity)::int AS ordered_units, COUNT(DISTINCT po.po_id)::int AS
orders
FROM suppliers s
JOIN purchase_orders po ON po.supplier_id = s.supplier_id
JOIN po_items i         ON i.po_id        = po.po_id
WHERE po.status <> 'CANCELLED'
GROUP BY s.supplier_id, s.name
HAVING SUM(i.quantity) > $2
ORDER BY ordered_units DESC;
```

11.5 Correlated subquery (NOT EXISTS): products never ordered

```
SELECT p.product_id, p.sku, p.name, c.name AS category
FROM products p JOIN categories c ON c.category_id = p.category_id
WHERE NOT EXISTS (SELECT 1 FROM po_items i WHERE i.product_id = p.product_id)
ORDER BY p.name;
```

11.6 Scalar subqueries: dashboard totals

```
SELECT (SELECT COUNT(*) FROM products)                                    AS products,
       (SELECT COUNT(*) FROM warehouses)                                  AS warehouses,
       (SELECT COALESCE(SUM(stock_value), 0) FROM v_stock_summary)        AS stock_value,
       (SELECT COUNT(*) FROM low_stock_alerts WHERE resolved = false)     AS open_alerts;
```

11.7 Purchase-order list with totals

```
SELECT po.po_id, po.status, s.name AS supplier, w.name AS warehouse,
       COUNT(i.product_id) AS item_count,
       COALESCE(SUM(i.quantity * i.unit_cost), 0) AS total_value
FROM purchase_orders po
JOIN suppliers  s ON s.supplier_id  = po.supplier_id
JOIN warehouses w ON w.warehouse_id = po.warehouse_id
LEFT JOIN po_items i ON i.po_id = po.po_id
WHERE po.status = $1
GROUP BY po.po_id, s.name, w.name
ORDER BY po.po_id DESC;
```

11.8 Recent stock movements (ORDER BY … LIMIT)

```
SELECT m.movement_id, p.sku, w.name AS warehouse, m.change_qty, m.movement_type, m.created_at
FROM stock_movements m
JOIN products p ON p.product_id = m.product_id JOIN warehouses w ON w.warehouse_id = m.warehouse_id
ORDER BY m.created_at DESC, m.movement_id DESC LIMIT 10;
```

11.9 MongoDB queries

```
db.product_specs.findOne({ product_id: 1 })                                    // specs for one product
db.product_specs.find({ category: "Electronics", "specs.max_wattage": { $gte: 45 } })  // query on a nested
field
db.supplier_documents.updateOne(                                               // add feedback, create the
doc if missing
  { supplier_id: 2 },
  { $push: { feedback: { po_id: 3, rating: 4, issues: ["late_delivery"], date: new Date() } },
    $setOnInsert: { certifications: [], contacts: [] } },
  { upsert: true })
```

12 System Architecture

Three tiers: browser, Next.js server (pages and API), two databases

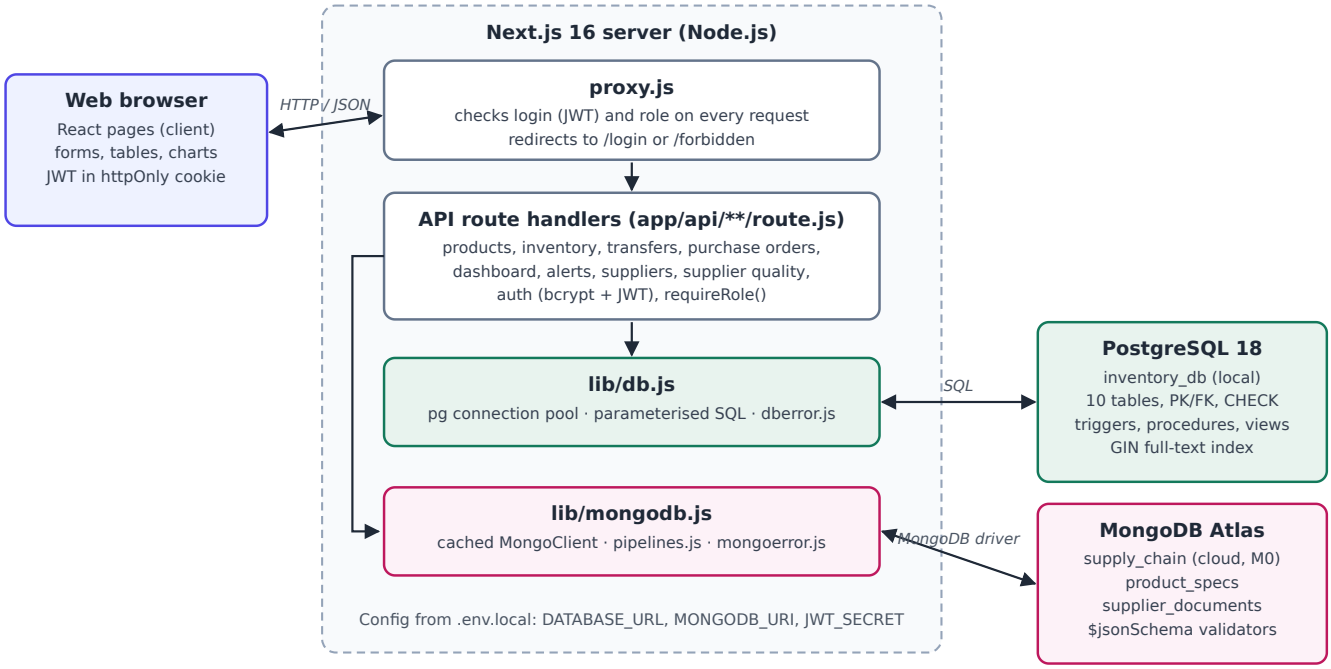


Figure 12.1: System architecture

12.1 Layers

Layer	Responsibility
Presentation	Next.js App Router pages (React client components) with Tailwind CSS and Recharts. Pages call the API with `fetch` and show database errors in a red alert.
Access control	`proxy.js` runs before every request: verifies the JWT cookie, redirects unauthenticated users to `/login`, and sends users without the right role to `/forbidden` (or returns 401 for API calls).
API	Route handlers in `app/api/**/route.js`. Each one calls `requireRole(...)`, validates its input, runs parameterised SQL or MongoDB operations, and returns JSON.
Data access	`lib/db.js` (shared `pg` pool), `lib/mongodb.js` (cached client), `lib/pipelines.js`, and error mapping that turns database errors into HTTP 400 responses with readable messages.
Data	PostgreSQL 18 (local) and MongoDB Atlas (cloud). Connection strings come from environment variables.

12.2 Example request flow: stock transfer

1. The manager submits the transfer form; the page sends `POST /api/transfers`.
2. `proxy.js` verifies the session cookie; the route checks the role is ADMIN or MANAGER.
3. The route runs `CALL transfer_stock($1, $2, $3, $4)`.
4. Inside PostgreSQL the procedure locks rows, checks stock and updates inventory. The `trg_capacity` trigger checks the destination, and `trg_low_stock` may create or resolve an alert. Everything commits together, or nothing does.
5. On failure, the PostgreSQL error message is returned as HTTP 400 and shown in the form.

12.3 Security

- **Passwords:** stored only as bcrypt hashes (cost factor 10) and compared with `bcrypt.compare`.
- **Sessions:** a signed JWT (HS256, 8-hour expiry) in an `httpOnly`, `sameSite=lax` cookie that JavaScript cannot read; `secure` in production.
- **SQL injection:** every query uses `$1, $2 …` placeholders. The only dynamic SQL is `ORDER BY`, which is limited to a whitelist of column names.
- **Credentials:** stored in `.env` / `.env.local`, which `.gitignore` excludes. The repository only contains `.env.example` with placeholders. After a real password was accidentally committed during

development, the Atlas password was changed and the file was cleaned.

- **Authorisation:** checked both in `proxy.js` (pages) and in every API route (`requireRole`). Suppliers are additionally filtered to their own `supplier_id` in every query, and they get the same "not found" response for another supplier's data, so they cannot probe IDs.

13 Application Screenshots

The Next.js interface, running against PostgreSQL 18 and MongoDB Atlas

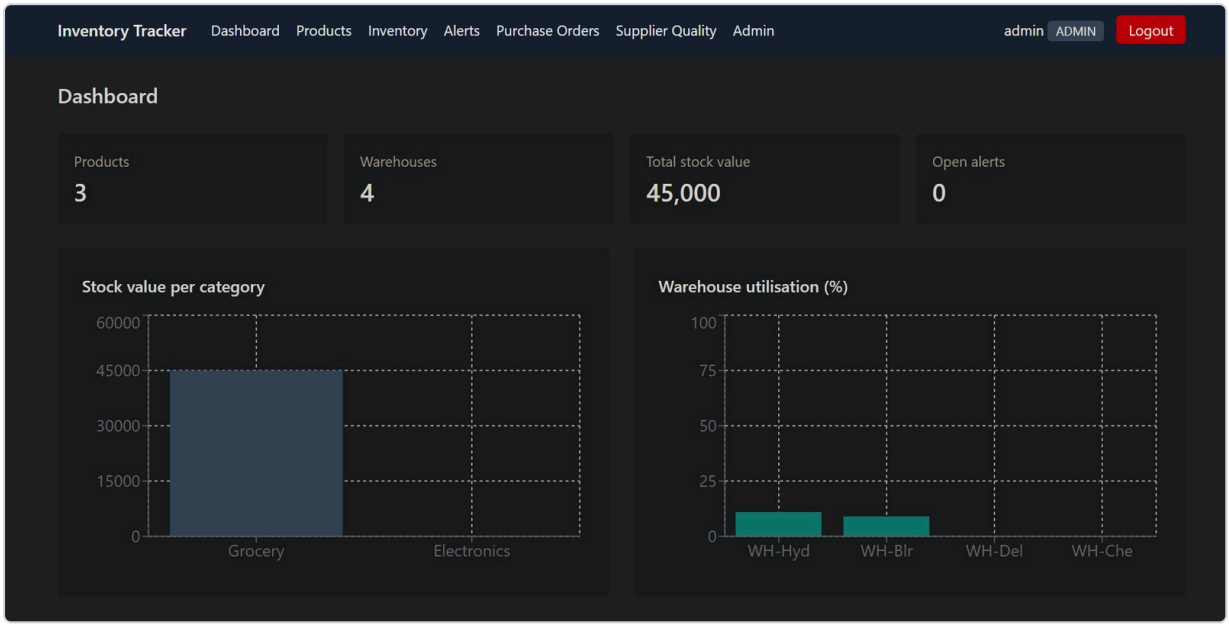


Figure 13.1: Dashboard: totals (scalar subqueries over a view), stock value per category (GROUP BY) and warehouse utilisation (LEFT JOIN with a computed ratio)

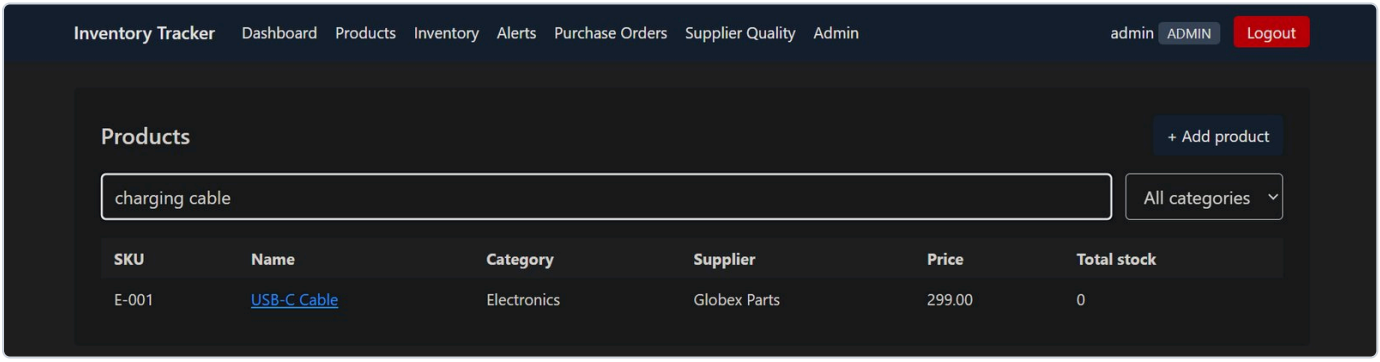


Figure 13.2: Full-text search: "charging cable" finds the USB-C cable through the GIN-indexed tsvector column

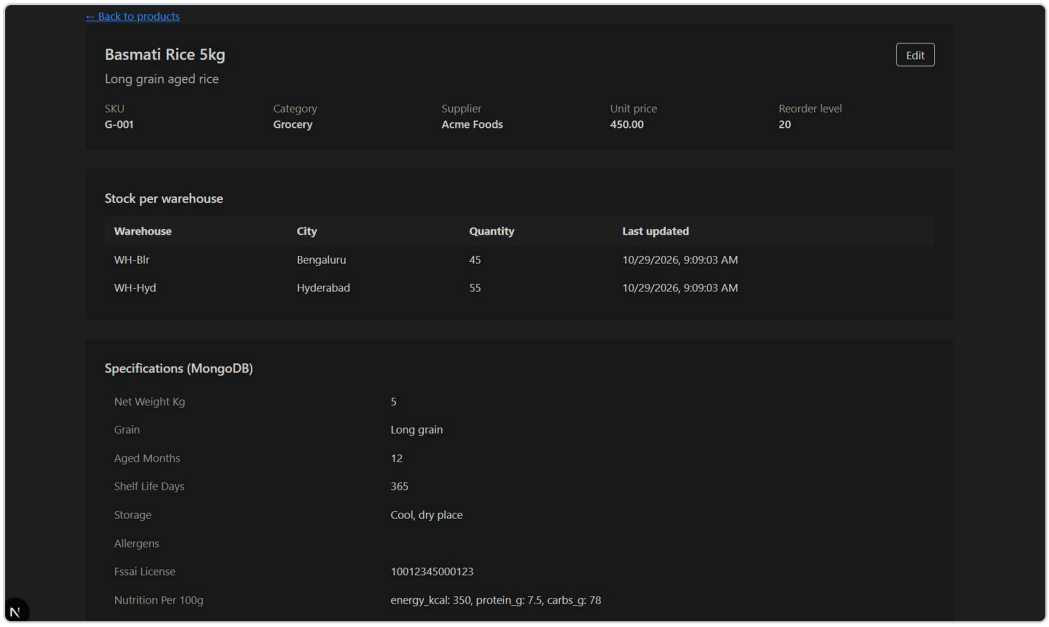


Figure 13.3: Product detail: PostgreSQL data (SKU, price, stock per warehouse) combined with category-specific specifications from MongoDB

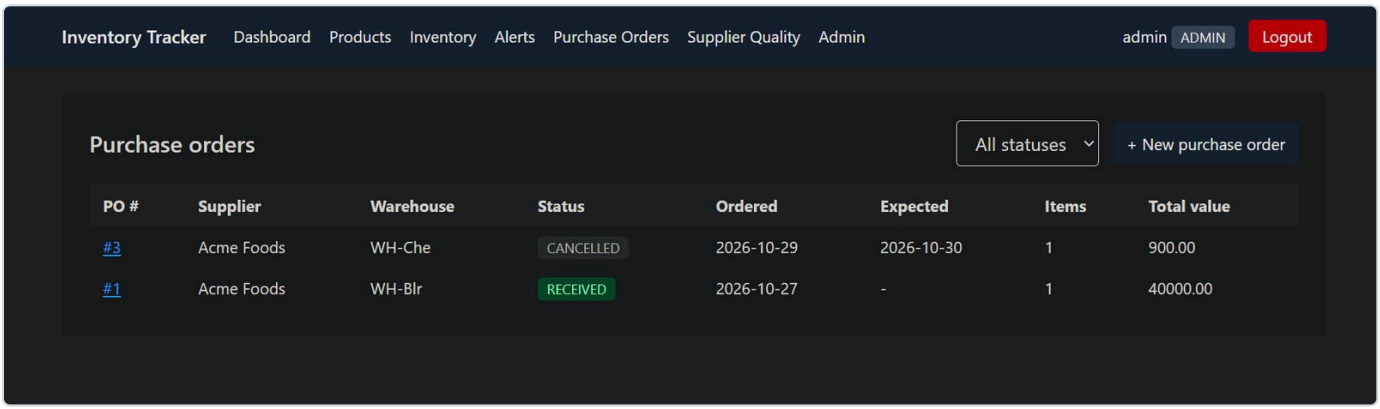


Figure 13.4: Purchase orders with totals (JOIN + GROUP BY + SUM). PO #1 was received through the receive_purchase_order procedure; PO #3 was cancelled.

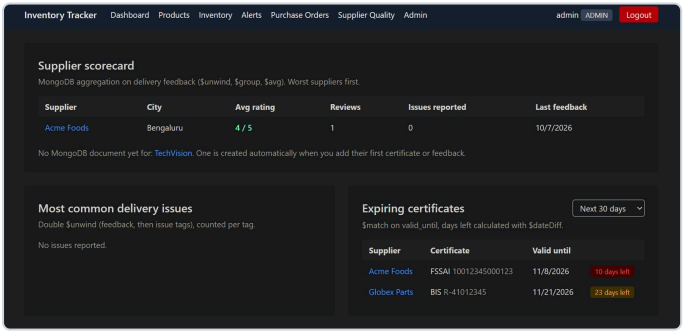


Figure 13.5: Supplier Quality: scorecard and expiring certificates from MongoDB aggregation pipelines; suppliers without a MongoDB document are listed

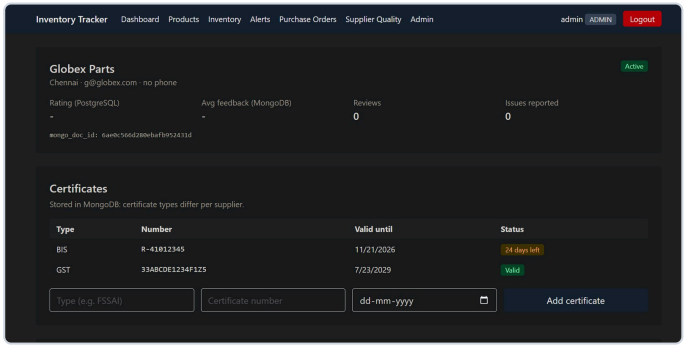


Figure 13.6: Supplier page: PostgreSQL supplier record and linked mongo_doc_id, with certificates stored in MongoDB

14 Testing Results

Database-level tests (psql), application tests (browser) and NoSQL tests

14.1 Test cases

ID	Test	Input / action	Expected result	Result
T1	Schema creation	Run `schema.sql`	10 tables, 6 indexes, 2 triggers, 2 views, 2 procedures created	**Pass**
T2	Receive PO (procedure)	`CALL receive_purchase_order(1)`	100 units added; PO_RECEIPT logged; status RECEIVED	**Pass**
T3	Low-stock alert (trigger)	Transfer 85 of 100 units out of WH-Blr (reorder level 20)	Stock 15; alert created with resolved = f	**Pass**
T4	Reorder view	`SELECT * FROM v_reorder_suggestions`	G-001 at WH-Blr, suggested quantity 25	**Pass**
T5	Alert auto-resolve (trigger)	Transfer 30 units back to WH-Blr	Same alert now resolved = t	**Pass**
T6	Insufficient stock	Transfer 500 units (only 45 available)	Error "Insufficient stock: have 45, need 500"; rollback	**Pass**
T7	Capacity exceeded (trigger)	Transfer 45 units into WH-Che (capacity 40)	Error "Warehouse 2 capacity exceeded (0 + 45 > 40)"; whole transfer rolled back	**Pass**
T8	Double receive (row lock + status check)	`CALL receive_purchase_order(1)` again	Error "PO 1 is already RECEIVED"	**Pass**
T9	Movement log and view	Query stock_movements, v_stock_summary	5 movements; G-001 total 100, value 45 000.00	**Pass**
T10	Full-text search	Search "charging cable" (psql and UI)	Returns E-001	**Pass**
T11	UI: insufficient stock	Transfer 999 units of G-001 from WH-Blr	Red alert with the procedure's message	**Pass**
T12	UI: capacity exceeded	Transfer 41 units to WH-Che	"Warehouse 2 capacity exceeded (0 + 41 > 40)"	**Pass**
T13	UI: UNIQUE constraint	Add a supplier with an existing email	"duplicate key value violates unique constraint suppliers_email_key"	**Pass**
T14	Role-based access	SUPPLIER user opens `/admin`	Redirected to "Access denied"; supplier menu shows only allowed pages	**Pass**
T15	MongoDB aggregations	`node mongo/aggregations.mjs`	Scorecard, issues, expiring certificates (days_left), spec fields per category, nested-field query	**Pass**
T16	Feedback rules	Feedback for an ORDERED PO, another supplier's PO, a duplicate PO, an unknown tag	Each rejected with HTTP 400 and a clear message	**Pass**
T17	MongoDB validator	Feedback with rating 7 or 4.5 (sent directly to the API)	Rejected by `$jsonSchema` (error 121) and shown as a validation error	**Pass**
T18	Upsert + link	Add a certificate for a supplier with no MongoDB document	Document created; ObjectId written to suppliers.mongo_doc_id	**Pass**

T1–T10: `db/seed_and_test.sql` (Figures 14.1–14.4). T11–T14: browser (Figures 14.5–14.8). T15: Figure 14.9. T16–T18: API tests during development of the supplier-quality module.

14.2 SQL test run



Figure 14.1: Running schema.sql: DROP SCHEMA … CASCADE, then CREATE / ALTER TABLE statements (T1)

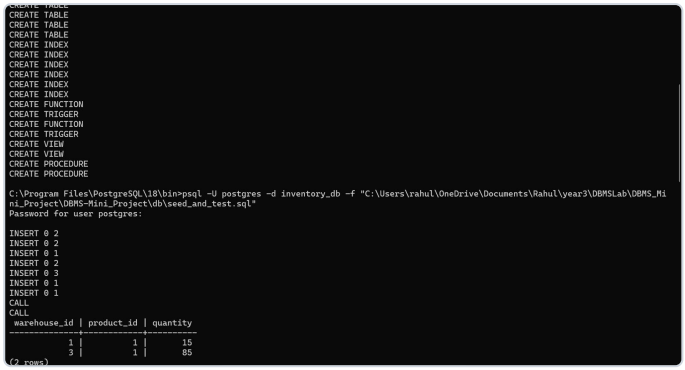


Figure 14.2: Indexes, functions, triggers, views and procedures created; seed data inserted; inventory after the receive and transfer (T1–T3)

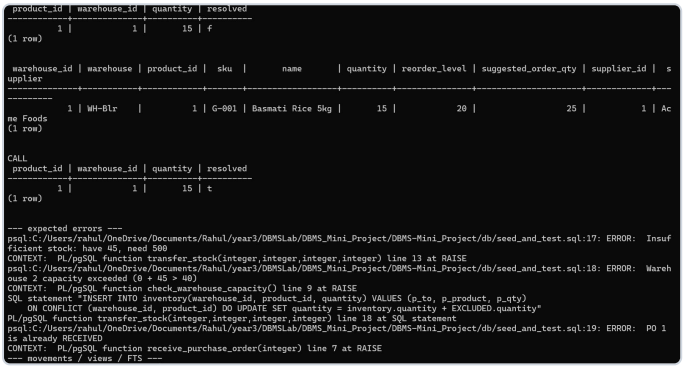


Figure 14.3: Alert created (resolved = f), reorder suggestion from the view, alert resolved (resolved = t), and the three expected errors (T3–T8)

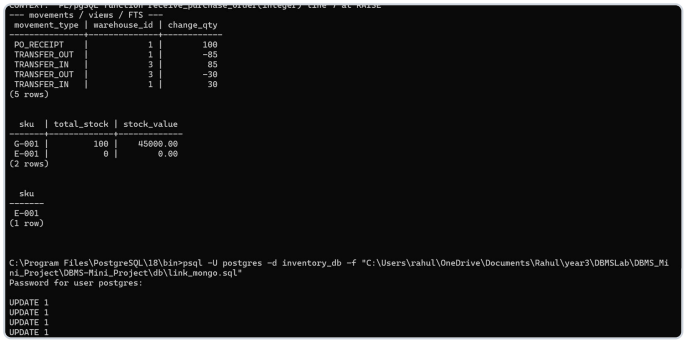


Figure 14.4: Stock movement log, v_stock_summary and the full-text search result; link_mongo.sql updates (T9, T10)

14.3 Application tests

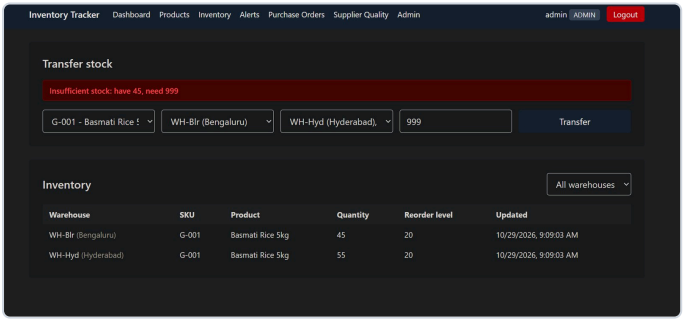


Figure 14.5: Insufficient stock rejected by the transfer_stock procedure (T11)

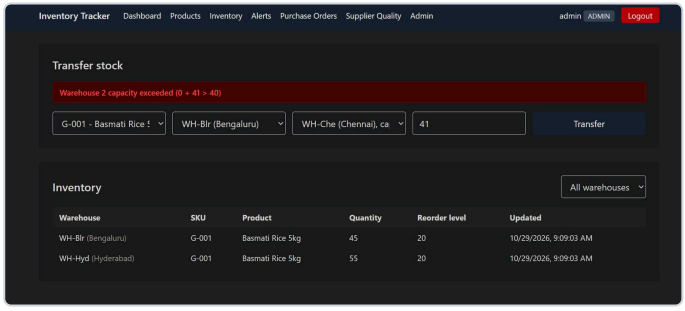


Figure 14.6: Warehouse capacity enforced by the trg_capacity trigger (T12)

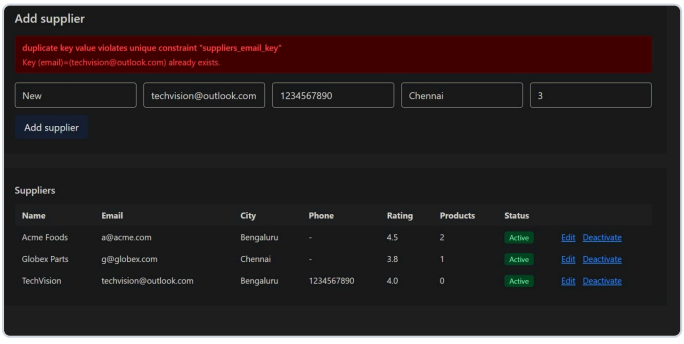


Figure 14.7: Duplicate supplier email rejected by the UNIQUE constraint (T13)

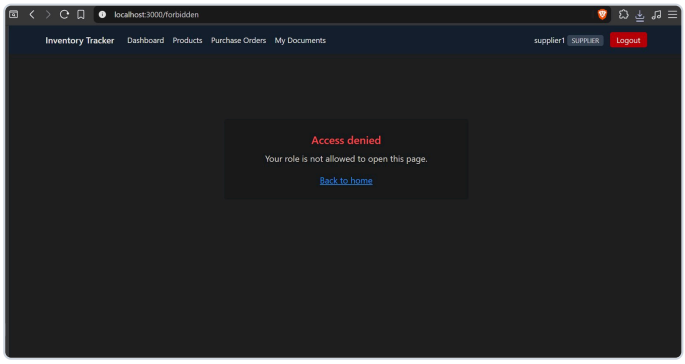


Figure 14.8: SUPPLIER role blocked from the admin area, with a reduced navigation menu (T14)

14.4 NoSQL tests

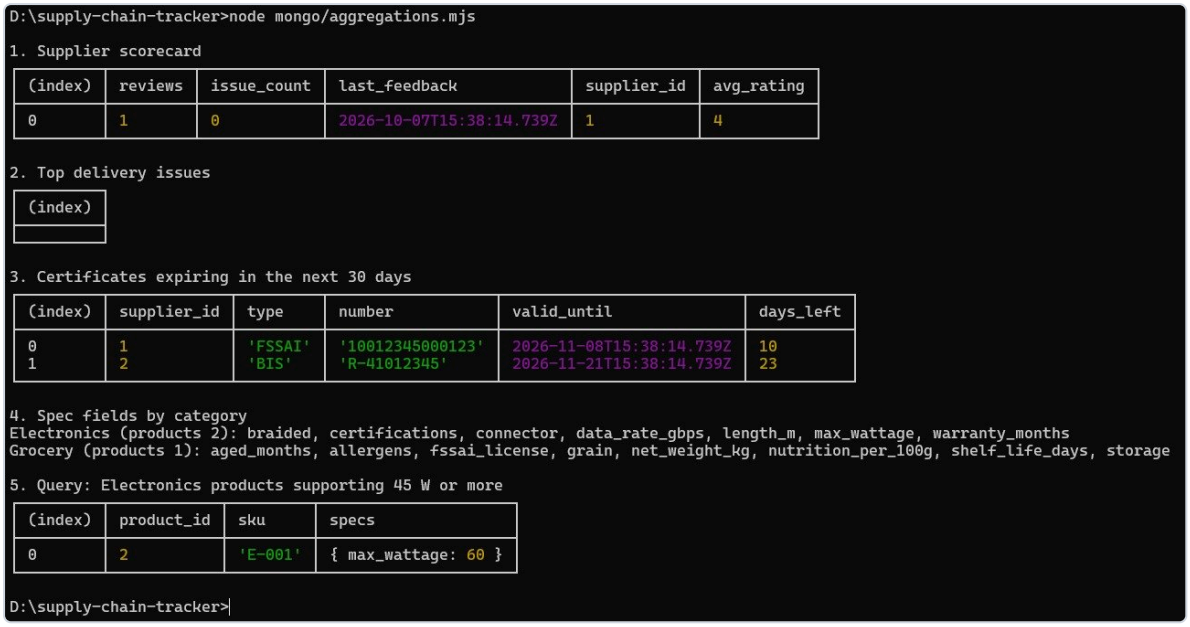


Figure 14.9: Output of the four MongoDB aggregation pipelines and the nested-field query (T15)

15 Individual Contributions

Both members worked on the database and on the application, as required by the brief

Member	Database	Application and other
Prithvish Adhikari	• PostgreSQL schema: tables, constraints, indexes • Triggers, stored procedures, views • Seed and test script • MongoDB Atlas setup, collections, `$jsonSchema` validators, indexes • Aggregation pipelines • PostgreSQL–MongoDB link script	• Supplier Quality dashboard and supplier documents pages • Feedback and certificate APIs (cross-database checks, upsert, rating sync) • MongoDB error mapping, feedback clean-up script • Production-build fix, project README • Credential clean-up and Atlas password change
Rahul Prasanna	• PostgreSQL integration: connection pool, parameterised queries • Analytical SQL for the dashboard (GROUP BY, HAVING, subqueries) • Transactional purchase-order creation • Database testing and the SQL test run	• Next.js project setup and structure • Authentication (bcrypt, JWT cookie), roles, proxy • Products and search, inventory and transfers, alerts • Purchase orders, dashboard with charts, admin pages • Application testing and screenshots

Individual commits are visible in the repository history. Problem analysis, the ER design and the choice of databases were done together.

16 Conclusion and Future Enhancements

Outcome of the project

16.1 Conclusion

The project delivers a working inventory and supply-chain system in which the important rules are enforced **by the databases themselves**. Warehouse capacity, non-negative stock, valid order states and unique identifiers are guaranteed by constraints and triggers. Multi-row operations run as transactions with row-level locking, so stock totals stay consistent even with concurrent users. Low-stock alerts are created and resolved automatically in the same transaction as the stock change.

Using two databases was a decision based on the data, not a technology demonstration. Transactional, highly related data lives in PostgreSQL. Product specifications and supplier documents, whose structure differs from record to record, live in MongoDB, where `$jsonSchema` validators still enforce the required fields per category and aggregation pipelines produce the supplier scorecards. The Next.js application connects the two through an API layer with role-based access, bcrypt-hashed passwords and parameterised queries.

Brief requirement	How it is met
SQL (mandatory) + NoSQL	PostgreSQL 18 + MongoDB Atlas (Option 1)
ER diagram → relational schema	Sections 6 and 7
CREATE, ALTER, DROP, keys, constraints, indexes, views	Section 9
INSERT, UPDATE, DELETE, SELECT; JOIN, GROUP BY, HAVING, ORDER BY, aggregates, subqueries	Section 11
≥ 4 advanced SQL features + 1 NoSQL feature	6 SQL + 2 NoSQL (Section 9.7)
Next.js interface, API layer, environment variables	Sections 12 and 13
Authentication, ≥ 2 roles	3 roles (Section 4.1, 12.3)
Validation at database level; no plain-text passwords	Constraints, triggers, validators; bcrypt
Git/GitHub with individual commits and README	Repository; Section 15

16.2 Future enhancements

1. **Semantic search over feedback** using pgvector or MongoDB Atlas Vector Search, for example "find suppliers with packaging problems" even when the wording differs.
2. **Demand forecasting:** use stock-movement history to predict reorder dates and quantities instead of a fixed reorder level.
3. **Sales orders and invoicing** with GST, so stock goes out as well as in.
4. **Notifications:** email or WhatsApp messages for low-stock alerts and certificates about to expire.
5. **Barcode scanning** on mobile for receiving goods and stock counts.
6. **Batch and expiry tracking** for perishable goods, with first-expiry-first-out picking.
7. **Supplier self-service:** suppliers upload certificates and confirm delivery dates from their portal.
8. **Audit log** of who changed what, and a **cloud deployment** with HTTPS and managed PostgreSQL.

A Appendix: Setup and Execution

How to run the project from the repository

Prerequisites: Node.js 20+, PostgreSQL 18 and access to the MongoDB Atlas cluster. Run from the repository root:

```
psql -U postgres -c "CREATE DATABASE inventory_db;"
psql -U postgres -d inventory_db -f db/schema.sql
psql -U postgres -d inventory_db -f db/seed_and_test.sql
psql -U postgres -d inventory_db -f db/link_mongo.sql
cd web
npm install
copy .env.example .env.local        // then fill in DATABASE_URL, MONGODB_URI, MONGODB_DB, JWT_SECRET
node --env-file=.env.local scripts/create-user.mjs admin <password> ADMIN
npm run dev                         // open http://localhost:3000
```